

Evaluating Rollback Networking Outside of
Peer-to-Peer Two-Player Games

Declan McGuinness

BSc (Hons) Computer Game Applications
Development, 2023

School of Design and Informatics
Abertay University

Table of Contents

Table of Figures.....	2
Acknowledgements.....	2
Abstract.....	3
Abbreviations and Terminology.....	4
1. Introduction.....	1
2. Literature Review.....	2
2.1 Lockstep Netcode.....	2
2.2 Rollback Netcode.....	3
2.2.1 Prediction.....	3
2.2.2 Simulating and Rolling Back.....	4
2.3 Functional Requirements.....	5
2.3.1 Determinism.....	5
2.3.2 Serialisation.....	6
2.3.3 Optimisation.....	7
3. Methodology.....	7
3.1 Overview.....	7
3.2 Research and Early Development.....	8
3.2.1 Choosing an Engine.....	8
3.2.2 Determinism in Unreal Engine 5.....	9
3.2.3 Floats, Physics and Time.....	9
3.2.4 Loading and Dynamic Spawning.....	12
3.3 Implementation.....	12
3.3.1 Replacing UE5 Physics.....	12
3.3.2 Initial Modifications to Bullet Implementation.....	13
3.3.3 GGPO Unreal Plugin.....	14
3.3.4 How to Implement GGPO.....	14
3.3.5 Implementing GGPO Into the FPS application.....	16
3.4 Application Testing Tools.....	17
3.4 Testing Methodology.....	18
4. Results.....	19
4.1 Overview.....	19
4.2 GGPO Log data.....	19
4.3 Unreal Insights Data.....	22
5. Discussion.....	24
5.1 Overview.....	24
5.2 Testing Data Analysis.....	24
5.2.1 Internal Simulation Performance.....	24
5.3 Reflecting on Rollback Development.....	26
6. Conclusion and Future Work.....	28
6.1 Improvements to be Made and Future Work.....	28
List of References.....	30

Bibliography.....	32
-------------------	----

Table of Figures

Figure 1 – Example of a Rollback occurring on Frame 4.....	5
Figure 2 – Rollback Logs Aggregate Data.....	19
Figure 3 – Total Quantity of Rollbacks During a Test.....	20
Figure 4 – Average Number of Frames for Rollbacks Overall.....	21
Figure 5 – Largest Single Rollback per Player per Test.....	21
Figure 6 – Unreal Insights Frame Timing Aggregate Data.....	22
Figure 7 – Unreal Insights Frame Timing Comparison in ms.....	23
Figure 8 – Frame Timing Comparison upon Conversion to Equivalent FPS.....	23

Acknowledgements

This project wouldn't have been possible without the massive help I've gotten along the way. Thanks first go to my supervisor Andrei Boiko, for helping guide me despite the detailed nature of my questioning. Next would be the wider open-source community surrounding the topic, as this project just wouldn't run without the multiple tools used in development. And finally, thanks to all the developers who simply won't stop preaching their knowledge of Rollback and determinism, thanks to you, the rabbit hole of information out there managed to claim another victim. Special mention to Jennifer Henry, as she took the time to answer some of my questions about her 2019 GDC talk about For Honor's Deterministic Simulation, which wasn't freely available at the time.

Abstract

Context: As online gaming boomed to the mainstream, it became evident that quality networking systems are necessary to remain competitive. In gaming, a good system is one that properly addresses the challenges of online connectivity, hence our focus on a new lag compensation technique known as Rollback networking.

Aim: To further the understanding of using Rollback outside of its fighting-game niche, identifying key challenges of development, and evaluating its performance.

Method: A basic FPS game demo was made in UE5, then networked with Rollback. Networking involved going from an industry-standard implementation to a vastly different result to accommodate Rollback's technical requirements. The resulting application was then tested under different conditions and the development approach was analysed to understand how Rollback affects the development of the game it's in.

Results: The final outcome revealed just how thoroughly the requirements for determinism and save-states affect wider development, and the gathered data highlights the previously unexplored impacts poor performance has on all players within the session.

Conclusion: This project demonstrates the strengths and weaknesses of Rollback technology and makes recommendations to aid in popularizing its adoption within the appropriate context. The study was limited in scope due to the high requirements but low resources available for this implementation. However, these limitations bore fruit in illustrating the final results.

Abbreviations and Terminology

FPS - Frames Per Second or First-Person Shooter depending on context.

GGPO - Short for “Good Game Peace Out”, the name for a Rollback SDK.

Rollback - Shorthand for Rollback Networking or Rollback Netcode.

UE# / Unreal - Shorthand for a specific version of Unreal Engine, or the engine itself.

1. Introduction

The idea of multiplayer games fundamentally changed with the popularisation of online gaming in the early 2000s with the original Xbox (2001) and World of Warcraft (2004). Since then — and especially in the last few years — online multiplayer has eclipsed local multiplayer both in popularity and availability. This is likely due to online play allowing for a greater amount of possible features, such as player counts in a single session being able to scale from a couple to hundreds; and allowing for competitive matchmaking systems that allow for developers to better group players with others of similar skill.

To achieve this, game developers have created various types of networking systems and infrastructure to support online play. These systems allow for game data to be sent between players across the internet instead of being limited to LAN connections. However, with the increase of scale came new technical challenges exclusive to online connections, the preeminent of which is latency and how it drastically varies for all players.

To resolve this, modern games employ different strategies and techniques commonly known as “lag compensation” to sync players' games and prevent the frustrations of desyncs. However, many solutions can come with their new problems, a common one seen even in recent FPS games is the phenomenon of being “shot around a corner” (Lee and Chang, 2017).

The focus of this project is on a specific state-of-the-art networking solution — pioneered in the fighting game genre — known as Rollback networking. This project aims to evaluate its feasibility outside of its niche, to see if it can be used more broadly. The main objectives are to understand the principles within current examples of Rollback development, then use that understanding to create an FPS game that uses the technology as a case study, and finally, evaluate its performance on the game to further understanding of this topic.

The project's scope will be limited to implementing the bare bones of an FPS game as the focus is on using the technology in a new genre, not creating a game in itself. The limited time available and single developer means that the result is best used as a case to study or an early example of the topic, and should be considered within context.

2. Literature Review

According to an interview with GameDeveloper.com (Orland, 2011), Rollback was pioneered by Tony Cannon with the invention of the GGPO SDK in 2006. He created the concept due to his frustration (shared by most players at the time) with the 'unplayable' online experience in early online fighting games. Unfortunately, this problem persists to this day; with the most recent example being Bandai Namco Studios' Super Smash Bros. Ultimate online gameplay (D'Anastasio, 2018). As per their official website, GGPO (no date) is described as follows:

“Designed specifically to hide network latency in fast-paced, twitch style games which require very precise inputs and frame perfect execution ... providing the illusion of a zero-latency network.”

Nowadays, with this approach's massive popularity amongst the fighting game community; its inclusion has become a selling point worth listing in new games' marketing material, for example: Guilty Gear -Strive- (ARCSYSTEMWORKS, 2020), and is getting added retroactively to older titles, for example: Guilty Gear Xrd -Revelator- and Xrd REV 2 (ARCSYSTEMWORKS, 2022).

2.1 Lockstep Netcode

Understanding the Rollback method first involves learning how its predecessor functions, as they follow the same fundamental principles. Also known as Delay-Based, it is the simplest way of handling latency in deterministic P2P games. The idea is to simply delay a local player's input execution by a few frames to account for an online connection's latency. Essentially, if a packet is expected to take a few frames worth of time to be received, then there needs to be an input delay to account for it. This delay is set up to be dynamic based on a rolling average of the player's ping, as online connections, especially over Wi-Fi, tend to fluctuate in

quality. Then if a packet isn't received within the grace (delay) period, perhaps due to a spike in the latency, the game will pause and wait for the packet and the rolling average input delay will increase accordingly (GDC, 2018).

While this system is extremely simple to implement since it's only a step above purely sending and waiting to receive packets, it comes with noticeable compromises to the smoothness of gameplay. The first issue is the possibility of stuttering due to repeated pauses from late packets. On top of impacting gameplay smoothness, these pauses also affect both players' play directly as any inputs done by players during a sudden stutter are not recorded by the game. Potentially causing fighting game combos to mess up since they rely on precise sequential timing.

The second compounding issue comes from the input delay being dynamic. A static delay would be easy for players to account for it or even not notice in the first place, but a dynamic one can noticeably change from match to match or even within the same one if there's a particularly unstable connection. So players have to deal with wildly inconsistent delay timings which reduce the competitiveness of online matches compared to an equivalent offline match.

2.2 Rollback Netcode

Rollback was created as an evolution upon Lockstep with the express goals of solving its problems and making online gameplay as smooth as offline. Instead of a naive 'delay, pause, and wait to receive late packets' approach Rollback uses a three-step process consisting of predicting, simulating, and rolling back.

2.2.1 Prediction

To start with there will always be a preset, static input delay, generally set to around 3 frames, with some games even including this as an option changeable by the player (Infil, 2019). Any packets received during this grace period would be executed just as with lockstep, with no additional steps needed. However, once the game hasn't received a packet on time Rollback's additional systems kick in starting with Prediction. The prediction's goal is to remove the need to pause gameplay by instead "predicting" what the remote player would have inputted and allowing gameplay to continue with this assumption. The original and most common

implementation this of prediction is to simply predict that what the remote player was last doing, is what they will keep doing; until told otherwise. This assumption tends to be correct >90% of the time as players are expected to input at most ~5 times per second, while games generally update at ~60fps (Hold Back to Block, 2019). There is ongoing research on improving this step using machine learning or “Hybrid Prediction” (Diler, Dalil and Mota, 2022) to more accurately predict input and reduce how often the rest of the rollback process is used, hence improving performance. If a prediction happens to be correct, then gameplay continues as normal, if not, then the rest of the process proceeds to the next steps, simulating and Rolling back.

2.2.2 Simulating and Rolling Back

Once the late packet with a new input arrives its data is used in a simulation. The simulation process consists of evaluating when the remote input should have originally occurred and simulating – based on recent savestate memory – how things should have played out in a perfect world. So within a single frame, the game engine will Rollback to the past, simulate gameplay with the new information, and then update the present with the now-corrected synced state. This only involves simulating the internal gameplay, and as such it doesn’t need to render while in progress, only the result at the end of the frame as usual.

Figure 1 demonstrates how GGPO does this process using an example of the remote player executing a fresh new input in frame 1, which was received late on the local machine during frame 4 and contradicted the prediction and necessitated a Rollback.

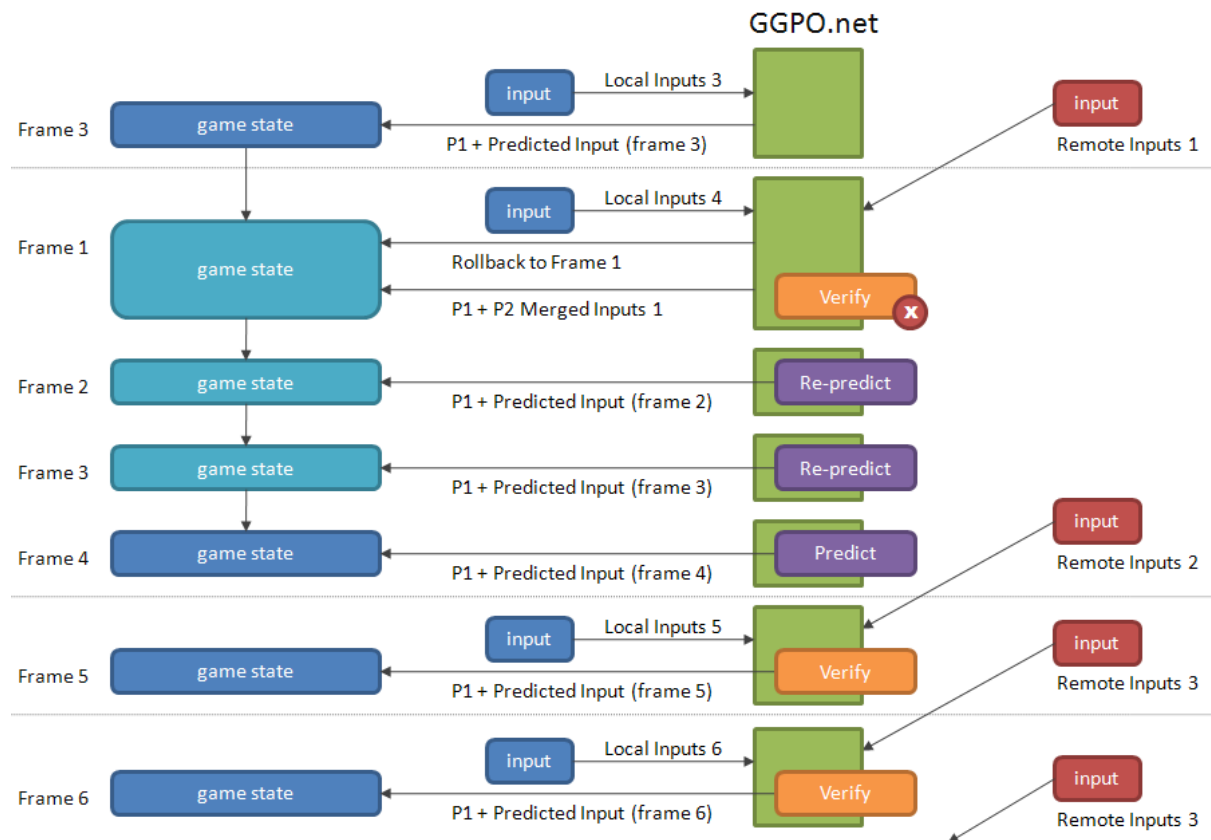


Figure 1 – Example of a Rollback occurring on Frame 4 (Cannon, 2019, diagram 4)

Altogether these steps remove the need for a dynamic input delay and pausing for packets, so both of Lockstep's problems are completely avoided and online gameplay's quality is drastically improved.

2.3 Functional Requirements

Before implementing Rollback networking into a game, it is important to fully consider the supporting systems implied in its functionality. These requirements are quite challenging to work with and are the main hurdle developers face when deciding to use the technology. While discussions of Rollback itself are quite easy to find among consumers, almost all of the literature and developer talks are about working within its requirements (GDC, 2019). Generally, they can be broken up into 3 main topics: determinism, serialisation, and optimization.

2.3.1 Determinism

Rollback only sends the player's inputs across the network, instead of replicating large quantities of gameplay data as you'd commonly find in other network

approaches (Unreal Engine, no date). This means that no machine has authority and is instructing others on the current synced game state. Because of this, the only way to keep all players synced is through deterministic simulation, commonly referred to as determinism. The concept of determinism in software is the principle that given the same situation, a process will always generate the same result without exception. Essentially, “with the same inputs, you always get the same outputs” (GDC, 2019). As long as this is respected, all player's games will independently arrive at synced gameplay. However, if there is even a single frame of non-deterministic gameplay, all is lost and a desync has occurred. This occurs because of the snowball effect; as every future frame is determined by this incorrect past, which leads to a larger and larger desync.

The foremost course of action is to remove the use of randomness or random number generators (RNG) within the gameplay, as it's implied to be a source of deviation. If necessary, it is possible to have pseudo-randomness, but only as long as it uses a pre-determined seed that was communicated to all devices or originates from the rest of the deterministic simulation. Avoiding randomness within gameplay scripts should be quite easy, but it does immediately rule out game genres that heavily rely on RNG, such as role-playing games, card games, and battle royales, from ever using Rollback. (GGPO, no date) Because of this, the project's gameplay will avoid RNG altogether to focus on more relevant challenges.

However, the most problematic sources of non-determinism are the ones outside of the usage of RNG. Jennifer Henry (GDC, 2019) highlights that developers “[shouldn't] assume that your engine is deterministic” and provides an example list of their most common sources of problems: floating point numbers, physics engines, time, and loading/dynamic spawning. As deterministic code is the topic that singlehandedly took the most time to research and tremendously impacts this project, the subjects within that list will be explained, when relevant, within the methodology and implementation sections.

2.3.2 Serialisation

As mentioned in section **2.2.2 Simulating and Rolling Back**, Rollback requires access to the immediate past for its namesake feature. This is done by keeping a

moving buffer of the dozen or so most recent frames of game state data. Just as with determinism, only the aspects that drive the gameplay need to be part of the simulation/game state, so non-essential components such as audio can be excluded.

While the specifics on how to best achieve this varies depending on the implementation (GDC, 2018; GDC, 2019; GDC, 2021), they all agree on a few general points. Those points are: the oldest state you need is determined by how far you need to roll back (generally ~10 frames in the past) and there is no need to save every redundant moment, so ideally it'd be optimized to keep only the non-redundant states within the Rollback window. The GGPO documentation describes that one of the simplest ways to accomplish this is by "... encapsulating all your game state into a single C structure. This both clearly delineates what is [within the] game state and what is not and makes it trivial to implement the save and load callbacks"(GGPO, no date). A slightly modified version of this approach is used in this project, as GGPO handles the saving & loading.

2.3.3 Optimisation

The final and most abstract requirement is good performance and optimization. Since the game needs to be able to simulate (without rendering) up to ~10 frames of gameplay within a single update — at least in the standard GGPO approach — there needs to be enough available headroom during a regular tick/update for the series of simulations. As a topic optimisation is very case-by-case, but many good examples of potential issues and suggestions are mentioned in some of the GDC talks (GDC, 2018; GDC, 2019). This does limit possible applications of rollback as some processing-heavy games or platforms with less processing power (such as mobile phones, browser games or the Nintendo Switch) often cannot afford to do these additional simulations.

3. Methodology

3.1 Overview

In order to analyze if Rollback can be used in a wider variety of games, this project requires an example game built using a modern game engine. As such, the application is a basic FPS game built in Unreal Engine 5 with the GGPOUE4 plugin

for networking. The initial approach was to fully utilize UE5's standard systems and attempt to plug it into the GGPO netcode, this was to identify if the workflow would be similar to Unreal's default networking. Further research and testing during development then proved this approach to be flawed, hence the adoption of a new physics engine, the Bullet3 Physics C++ SDK, and a more GGPO-centric design. Once complete, quantitative testing of the application's performance under different network conditions to test how this approach affects the application's performance.

3.2 Research and Early Development

While the initial plan was always to create a simple FPS game and network it with the GGPO SDK, the decision on how to approach this wasn't clear. So the first step in development was to research what game engines have been used alongside GGPO or other implementations of Rollback, both in and outside of fighting games, and if their decision was influenced by it.

3.2.1 Choosing an Engine

The most comprehensive non-editorial list of games using the technology is within the Wikipedia page of GGPO (no date), it contains lists of both games supporting GGPO and custom implementations of Rollback. If we ignore games that were originally released for arcade cabinets and were released in the last decade (2013 onwards), we get the following results at the time of analysis (May 27, 2023). From the GGPO list, most use unknown/proprietary engines, while 3 games are using Unreal Engine 4 and another 3 use Unity. Within the broader list, we see a similarly large quantity of unknown/proprietary engines, with 3 Unity games and an increase in Unreal usage: 5 in UE3, 8 in UE4, and 1 upcoming in UE5. We can especially see that Unreal dominates the upper market as every "Fighting Game of the Year" award winner by The Game Awards (2014 to 2022) use it alongside Rollback, the only exception being the *Super Smash Bros.* series in 2014 and 2019, who do not use UE or any form of Rollback. The last relevant fact within is of the sample, both of the non-fighting games using Rollback, *For Honor* and *Spelunky 2* (GDC, 2019; GDC, 2021), happen to use proprietary engines. The creator of *Spelunky* has publicly explained their reasoning for this decision, as quote: "*Spelunky 2* is using a custom

engine built by @BlitWorks. For what we're doing with that game, I think it's better for us to have complete control over the internals of the engine" (Yu, 2019).

Developing an entire engine on top of the needed game is outside of the scope of this project, as there simply isn't enough time and resources for such a task, Unreal Engine seems like the best choice at hand. This project specifically chose UE5 as its contemporaneous with future Rollback game development.

3.2.2 Determinism in Unreal Engine 5

After deciding on the engine, it's paramount to analyze what aspects of it need to be addressed to achieve determinism. Given the avoidance of RNG, the main aspects to highlight, as previously mentioned in the Literature Review, are floating point numbers, physics engines, time, and loading/dynamic spawning.

3.2.3 Floats, Physics and Time

A substantial quantity of the literature on determinism in games highlights the usage of floating point numbers and time within game physics as compounding problems (Dawson, 2013; Portelli, 2016; De Ioris, 2021). It all starts with the game world revolving around an Update/Timestep/Tick with a variable delta time. This delta time allows for the game to simulate a fluctuating framerate, with superior hardware allowing for higher definition. Now within regular games, this isn't an issue but is incompatible with both determinism and Rollback's frame-dependant resimulations.

This issue with delta time originates from the limitations of floating point numbers and how they impact anything they touch, especially physics. Float point numbers are limited in the specific decimal numbers they cover and any calculation using them will get a different result if the starting inputs vary whatsoever, even if the calculations would normally give the same result following conventional math (Dawson 2013). Boiling it down with an example and explanation from (De Ioris, 2021):

" [example:] $0.022211 + 0.01 = 0.032211000000000004$... the way floating point numbers work (under the hood) we can easily lose precision and introduce errors at a pretty fast pace. There are ways (well, tricks) to reduce that impact, but if

you do not have a stable numeric source (like a fixed sample rate), fighting it quickly becomes a lost battle.”

So over time, compounding inaccuracies in two simulations can give different results even when covering the same total time. De Ioris recommends the usage of Unreal’s FrameRate management system, through the usage of a (pre-determined) fixed frame rate or substepping: cutting time into fixed smaller divisions, and simulating multiple divisions instead of a larger chunk of time (2021).

As the surrounding literature and most of the documentation were referring to UE4, some tests were required to validate the claims and if the proposed solutions would still serve. The test is quite simple, inside a UE5 map a sphere was placed in the air above a triangular ramp with gravity enabled and a 5-second timer before the simulation resets. Then every frame logs the frame number, the total time passed, and the sphere’s XYZ position. The goal is to tweak the UE frame settings until there are the same positions over time, multiple times in a row (hence determinism) despite different conditions (target frame rates, computer load).

The testing methodology involved setting the conditions then running the same test 3 times in a row and comparing the logs to each other, then after testing all conditions see if tests under different conditions are identical. Tests were run using 4 different Unreal settings: 1- default, 2- fixed FPS of 60, and 3- substepping with a target of 60 FPS 4-combine settings of 2 and 3; then a second batch was run under “poor conditions”, which was emulated by having a heavy process load (in this case the game *Warhammer 40,000: Darktide*, which was quite unoptimized at the time) running in the background. The summarized results are listed below.

Test#1 - Both time and position vary in every single instance across conditions.

*Test#2 - All **in-game times** and positions are identical for all batches.*

Test#3 - Both time and position vary in every single instance across conditions.

*Test#4 - All **in-game times** and positions are identical for all batches.*

These results were different from what was initially hypothesised from the research. Starting with the obvious failures, test#1 was as expected as default UE5 is indeed

non-deterministic, however, test#3 was expected to be deterministic as it should be using fixed internal substeps. Further research into substepping revealed the probable cause was due to the logging occurring in the actor's Tick() function, which is called alongside the renderer, which doesn't substep (Portelli, 2016). Hence while the physics might be correct within the simulation, any interactions coming outside of it (including observing it) break the determinism, so for a passive simulation, this could be fine, but any gameplay done in the tick would make this irrelevant.

As for tests #2 and #3, looking at the data initially seems fine, as the logs have identical values, but further analysis highlights a new problem with 'human time', referred to as the "Wall Clock" in (De Ioris, 2021). Unreal's fixed framerate setting functions fine when it acts as a limit to the FPS, where it can simulate 60 frames within a real-time second, but once the application dips below the 60 FPS target. When this happens the in-game universe still updates at a sixtieth of a second each frame, but over longer spans of human time (ex: 60 frames per 1.2 real-time seconds). The resulting effect is not a problem in singleplayer games, as the simulation might run a bit slow but is still deterministic, however, in multiplayer this is a huge problem, as it would be possible for machines to 'outrun' each other if one ever has a dip below 60 Hz, meaning a player could be 'playing from the past'.

Combined with the revelation of the previously mentioned tests, it seems that standard usage of UE5's default updating/ticking isn't deterministic. Ideally, control over its tick as described in the article by Fiedler (2004) would solve this problem, this could potentially be done through a detailed look at UE4 PhysX SubstepTask functionality (Unreal Forums, 2016; Portelli, 2016). Unfortunately, these both rely on UE4's NVIDIA PhysX implementation, which was deprecated and replaced by UE5 Chaos Physics. This created a fork in the road; the first option is to change the engine to UE4 and redo the tests after setting up a new tick, which may or may not work out; or keep using UE5 and work around the problem by implementing a whole new physics engine that allows for manual ticking. This project went with the second approach, the reasons for which will be elaborated on when discussing its implementation in section **3.3.1 Replacing UE5 Physics**, but either approach should be viable.

3.2.4 Loading and Dynamic Spawning

There is a chance that determinism breaks when a scene is loaded or objects are dynamically spawned in some physics engines, including UE4's PhysX. UE5 exposes the `bEnableEnhancedDeterminism` setting to help with this (Unreal Engine, no date), but according to the documentation, this relates to the UE4 PhysX setting `eENABLE_ENHANCED_DETERMINISM` (NVIDIA, no date). As it is hard to know what's wrong between the documentation misleading its purpose or the application leaving in an unlabeled potentially deprecated feature, this setting is turned on just in case. Additionally, the application attempts to avoid this issue entirely by purely spawning actors at the start in a preset order (hence no dynamic spawning) and does not rely on UE5 physics.

3.3 Implementation

Now that a game engine has been chosen and its main deterministic hurdles have been identified, work on implementing the application can begin.

3.3.1 Replacing UE5 Physics

Given the conclusion of section **3.2.3 Floats, Physics and Time**, the project requires a drop-in replacement that allowed for manual control of the timestep. The solution first came to light through a GDC talk by Jared Cone about the physics and networking of Rocket League. He explains their decision to use the Bullet Physics Engine as it allowed them to have a deterministic physics engine using a fixed tick rate independent of the renderer. Additionally, he mentions how the process of integrating a new physics engine wasn't actually that difficult, as they can simply add in the parts of Bullet they need without touching UE3's physics, and then use it to simulate the gameplay physics exclusively. While not explicitly an implementation of Rollback, Rocket League's networking system follows many of the underlying principles around input-driven deterministic netcode, prediction, and re-simulation. These factors solidified the decision to use Bullet3 Physics for this project.

As for implementing Bullet into UE5, this project used Steve Streeting's approach to "Using Bullet for physics in UE4" (2020). Mostly, the guide is self-explanatory and functions identically within UE5. As stated by the developer, it is important to note however that this source serves best as a guideline and not copy-paste tutorial: "[t]his

implementation] is a specific solution to a specific problem, and requires some understanding of the situations it's applicable to. So the self-assembly presentation is intentional". Streeting's methodology can be summarised as creating a *PhysicsWorldActor* which acts as a container holding the entirety of the Bullet simulation. Once given a list of Unreal actors/meshes, it extracts the relevant colliders and creates invisible Bullet counterparts for the gameplay. Importantly, this *PhysicsWorldActor* only uses a fraction of the overall Bullet3 C++ SDK, meaning if additional functionality is desired developers simply need to include the relevant (C++ only, PyBullet isn't in this implementation) components of it.

3.3.2 Initial Modifications to Bullet Implementation

After doing a few more steps of setup, essentially adding uses of functions that were implemented without usage in the guide's overview, comes using it for gameplay purposes. The guide only covers how to create a 'passive' world purely made up of static (non-moving objects like walls) and dynamic (objects that react to gravity and the world around them) rigid bodies and leaves gameplay up to the user.

At this stage, it is relevant to highlight that GGPO networking hadn't been implemented yet, so the usage of Bullet followed a similar approach to using standard UE5 systems. The idea was to create an FPS pawn character that uses the new *Enhanced Input Plugin* — as it's the replacement for the deprecating default UE4 input system — to drive gameplay and use Bullet instead of UE5's default *ACharacter*. This was done initially by having the pawn request its addition to Bullet inside the *PhysicsWorldActor*, then translating the pawn's inputs into action within Bullet. For the movement, the translation involved supplying the input to the pawn's character controller, and then consuming the input in the `Tick()` function and converting it into a Bullet velocity and applying it to the pawn's rigid body. As for shooting, a simple Bullet ray cast was used, implementing this closely resembles the standard UE approach. This concluded the first iteration of using Bullet and the player pawn; if the project was purely about using Bullet within UE5, this approach was simple, functional and would do the job. However, once it came time to integrate GGPO, many overhauls would have to take place to account for unforeseen problems.

3.3.3 GGPO Unreal Plugin

After creating the first iteration of the gameplay, it was time to integrate the GGPO SDK into UE5. Thankfully, there is an unofficial plugin of it called GGPOUE4, made by BwdYeti (2020), that takes care of getting GGPO into the UE5 project. Just in case, it was first added to a clean UE5 project to make sure it would compile, and then that version was transferred into the main project files.

3.3.4 How to Implement GGPO

Now that the known prerequisites are out of the way, the gameplay was ready to be networked using GGPO. The main sources of information that guided the implementation were the official GGPO documentation (Cannon, 2019) and an unofficial UE4 port, made by BwdYeti (2020), of the official GGPO sample application *Vector War*.

To understand how this project will implement GGPO in an FPS game, it is best to first understand its existing implementation in the aforementioned *VectorWarUE4* port, as it illustrates the core pillars of Rollback development. The first pillar, separating the internal game simulation from the rest of Unreal, is immediately pointed out in the port's description:

“Unreal Engine is not deterministic, which is required [...] *VectorWarUE4* gets around this by running the original *VectorWar* code as its own game state, separate from the UE game logic. It only uses UE for reading player inputs, maintaining the update tick frequency, and rendering the game state by matching Unreal actors to the game state objects they represent. Similar solutions will be required for your own projects.”

This idea is what implementing *Bullet* was all about having access to an independent deterministic simulation that is represented on screen by the game engine.

Previously cited literature also echo this principle (GDC, 2019). Outside of preserving determinism, this has the additional benefit of encapsulating the simulation data for easy serialization. In practice, the GGPO documentation recommends having the game state as a singular C struct, as this makes it trivial to save and load copies of it (Cannon, 2019). The *VectorWarUE4* port follows this principle by having *GameState.h* as a regular public C++ struct, completely separate from the UE5

UObject systems. Now given its public nature, Unreal representations of the gameplay simply update their properties (such as position, velocity, etc...) of every render tick using their counterparts in the current internal game state, without making any changes. Other than the data, the internal *GameState.h* has all of the functions required to progress gameplay (Init, Handle Input and Update).

Separately from this, an Unreal GameStateBase class is in charge of implementing the GGPO interface and its *GGPOSessionCallbacks*. This GameStateBase is the central point managing both the internal simulation and the external Unreal components. It starts by creating the session and connecting the players to each other, once all connections are set, the gameplay is then initialized and begins running. GGPO then manages the internal game state through its callbacks inside GameStateBase. The most relevant callbacks are the Save, Load, and Advance Frame functions. Save and Load simply provide GGPO with a copy of the relevant frame's internal game state, while Advance Frame is the tick function used by the Rollback process. In practice this is how it works:

1. Save the present frame's game state (pre-tick). By default, GGPO saves the 10 most recent frames while deleting the oldest one once they become the 11th frame.
2. Start a rollback if necessary.
 - a. Load the relevant past frame.
 - b. Advance it once.
 - c. Save it as frame# load+1.
 - d. Repeat until caught up to the present frame.
3. Get the local player's input into GGPO. Note that due to the preset input delay, this input will be used in a future frame (2-frame delay used).
4. Advance the present frame once using GGPO's inputs (local + remote players inputs). The remote inputs could either be confirmed or predicted, but GGPO takes care of that process by itself. The internal game state's update is called using GGPO's input as a parameter.
5. Frame is complete, repeat these steps every frame starting at step 1.

3.3.5 Implementing GGPO Into the FPS application

The overall architecture in the project follows the methodology put forward by *VectorWarUE4*, as the basic setup of the GGPO session is the same no matter the game genre. Instead, the challenges arise from how the FPS game handles input and physics-based gameplay.

Fighting game inputs are generally mapped to explicit buttons, even joystick movement is interpreted as if it was a directional pad (up, down, left, and right). This means that the input can be written to an int32 bitmask, with each bit representing a button's state (pressed or not-pressed). Because of this, GGPO was designed to take in only up to 18 bytes of data for all players' input according to GGPO's `game_input.h` code (Cannon, 2019). This means each individual input must fit within 36 bits or 4.5 bytes. This creates a problem for our FPS game as the input was 21 bytes large: 3 floats for movement direction, 2 floats for DeltaMouse movement, and 1 bool for firing. While modifying GGPO to accept larger input data sizes could potentially work, as the creator stated it should work theoretically but hasn't been tested (GGPO, 2020), this development timeline required a faster solution. Instead of increasing the accepted size to fit the input, the input was reduced to fit within the default int32. First, the movement and firing were changed to be the first 6 bits of the input (4 movement directions, jump, fire). Second, came the mouse movement, UE5 registers mouse movement as 2 floats XY, each with a range of [-100, 100]. To fit these into the remaining 26 bits, they were first divided by 100 to reduce them to the [-1,1] range, then multiplied by 4096 and cast to an int, this int now has a value within [-4096, 4096] which can be stored within 13 bits as if it was an int13. Applied to both the mouse X and Y, we get 26 bits for mouse movement. All that's needed then is for the opposite process to occur when reading the input from an int32 within the internal game state.

As for the physics, the data needs to be within the game state C struct and be serializable. A kinematic fighting game could simply store the positions and rotations of objects, but Bullet3 is much more complex than that. The initial thought was a Bullet feature discussed on their forums, however, it turns out that this feature was in development and ended up being added to the PyBullet SDK in Python (PyBullet,

2020), while the code C++ systems have deprecated partial systems for it. This project attempted a few different approaches to serializing the game state Bullet data manually, however, the Bullet3 architecture didn't allow for them. The current implementation ended up making use of the partially implemented *btRigidBodyFloatData* left over within the partial C++ *btRigidBody* serializer. This data is used to Get or Set the *btRigidBody* properties when saving or loading. This approach arose from the project's limitations and could be a source of desyncs as it might not have everything Bullet needs for deterministic state restoration. This concludes the development of the applications' key features.

3.4 Application Testing Tools

Completing the application achieved the first half of this project's aim, specifically how implementing Rollback and its sub-systems affects the overall development process. Now the second half, how Rollback affects performance under different network conditions, can be evaluated.

To aid in testing, detailed logging was added to each step in the rollback process. To start, each GGPO callback has a log statement containing the local player ID and relevant frame number. This is to keep track of GGPO Rollbacks. Then at the end of each simulation frame, a detailed log of all player's states and input is made. As each machine logs the state of the entire local simulation, they can be compared with each other after the fact to look for desyncs.

As for measuring application performance, UE5 has a built-in feature called "Unreal Insights" which collects all UE5 analytics during runtime. This tool is limited as it can only see UE5-specific functions, so the internal simulation and GGPO aren't listed. This isn't much of a problem however as the detailed logs can be used to timestamp these non-UE5 systems within the analytics.

Finally, the *clumsy 0.3* network tool will serve in artificially creating different network conditions for each test. Specifically usage of the *Lag* feature, which can add latency to specific players. It's important to note that according to the clumsy manual, the

Lag feature likely increases latency by much more than just what the setting is set to due to how latency is measured (clumsy, no date).

3.4 Testing Methodology

Given the large quantity of data generated by each test, testing was limited to 4 tests each with different network conditions. All tests followed the same overall configuration:

- 2 Gaming Laptops were networked over an ethernet cable.
- Laptop A ran a single instance of the game, Player 3.
- Laptop B ran two instances of the game, Player 1 and 2.
- Both Laptops had clumsy 3.0 enabled during every test.
- Once the 2-minute timer began, each player would press “Start Game” (in order of 1->2->3).
- During play, players were essentially doing random inputs. Note that as 2 instances were running on Laptop B, they would manually intermittently between the players, controlling a single player at a time.
- After 2 minutes, all instances were closed and detailed logs for all 3 players were saved.
- Unreal Insights was used during every session on Laptop A (Player 3), as to function it requires the fully built custom UE5 engine code. Re-building the engine on multiple machines wasn't possible during the time allocated for testing, hence its usage exclusively on Laptop A.

The only changes between tests were to the network conditions. The following clumsy 3.0 settings were used on each test:

1. Default network conditions, no changes using clumsy.
2. Player 1 had no changes. Player 2 had 100ms of lag added to inbound and outbound packets. Player 3 had 150ms of lag added to inbound and outbound packets.
3. Player 1 had no changes. Both Players 2 and 3 had 200ms of lag added to inbound and outbound packets.
4. Player 1 had no changes. Both Players 2 and 3 had 300ms of lag added to inbound and outbound packets.

4. Results

4.1 Overview

The following results summarise the most relevant performance data of the application during the tests. Note that while 4 tests were done, data could only be generated by the first three tests, as the lag from test #4 was too great for a multiplayer connection as the session couldn't start. Another point unaddressed by this data is the occurrence of desyncs within the application build, these desyncs are likely caused by some non-determinism somewhere in the code. While this creates massive problems for functionality/playability, desyncs do not impact performance as the application itself is inherently unaware of the issue.

4.2 GGPO Log data

The logs contain the data for the GGPO processes and the internal game state. While the game state data is mandatory for debugging and desync detection, the following results are instead focused on the GGPO Rollback function calls.

Test #1				
Player	Total Frames	# of Rollbacks	Avg Rollback Frames	Largest Rollback
Player 1	3659	228	2.629	7
Player 2	3665	98	3.326	6
Player 3	3665	13	1.692	3
Test #2				
Player	Total Frames	# of Rollbacks	Avg Rollback Frames	Largest Rollback
Player 1	1376	280	2.985	7
Player 2	1382	298	2.865	7
Player 3	1377	226	2.339	6
Test #3				
Player	Total Frames	# of Rollbacks	Avg Rollback Frames	Largest Rollback
Player 1	1452	235	3.829	7
Player 2	1452	196	4.078	7
Player 3	1445	112	2.054	5

Figure 2 – Rollback Logs Aggregate Data

Each test went on for around 2 minutes (plus or minus a few seconds for the player to quit) with a fixed intern simulation tick speed of 16.66ms, which results in 60 Hz/FPS. This means that the simulation, and by extension the log data, should have a target of ~7200 total frames (60fps * 120 seconds) for all players on all tests. Other

than this target, the total number of frames isn't relevant when comparing players within a test, due to the margin of error of starting/quitting gameplay, but comparing the total numbers between tests shows how the other data points could impact performance.

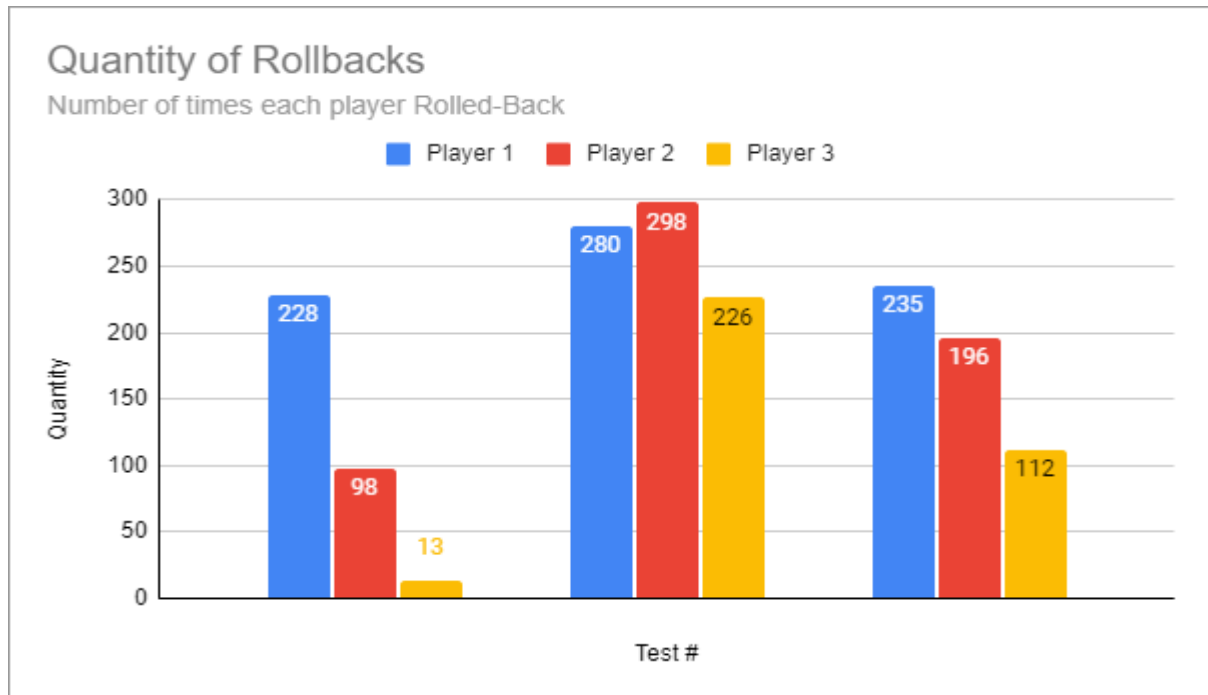


Figure 3 – Total Quantity of Rollbacks During a Test

The quantity of Rollbacks is measured by looking at the number of load_game_state callbacks. The size of a Rollback is measured by calculating the difference between the current frame to-be-advanced, and the frame just loaded, for example: “Saved Frame 100; Loaded Frame 97” results in a 3-frame Rollback as it would require advancing from 97 (loaded) -> 98 -> 99 -> 100 (present).

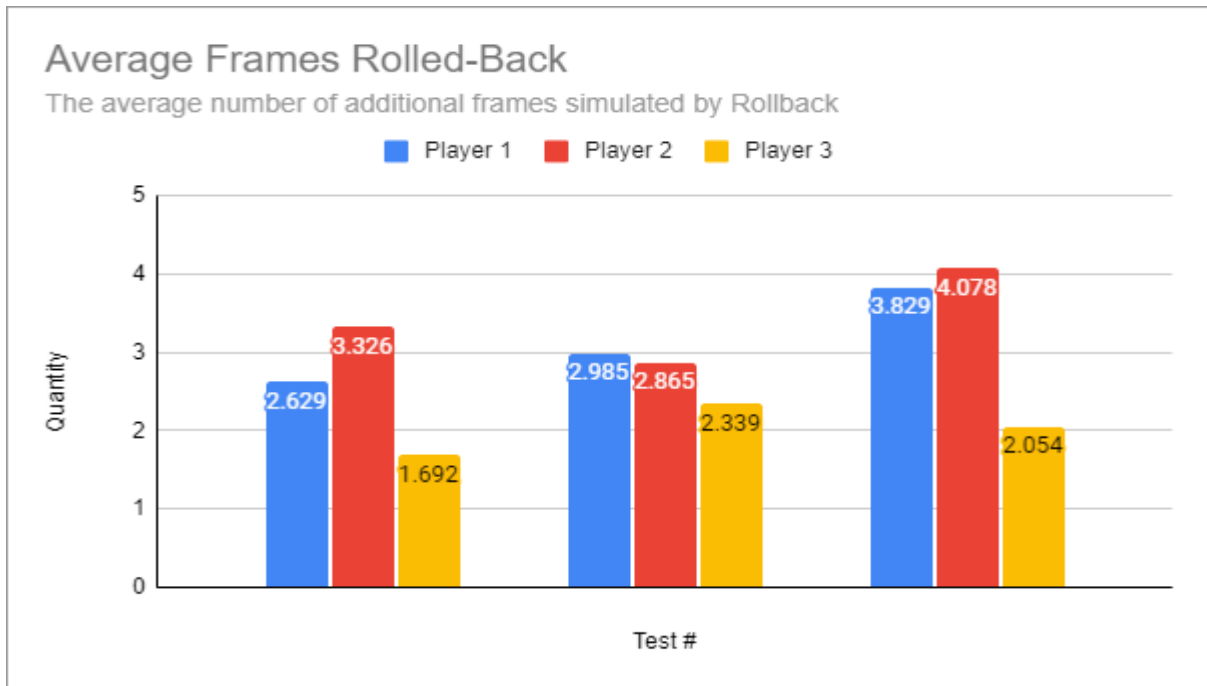


Figure 4 – Average Number of Frames for Rollbacks Overall

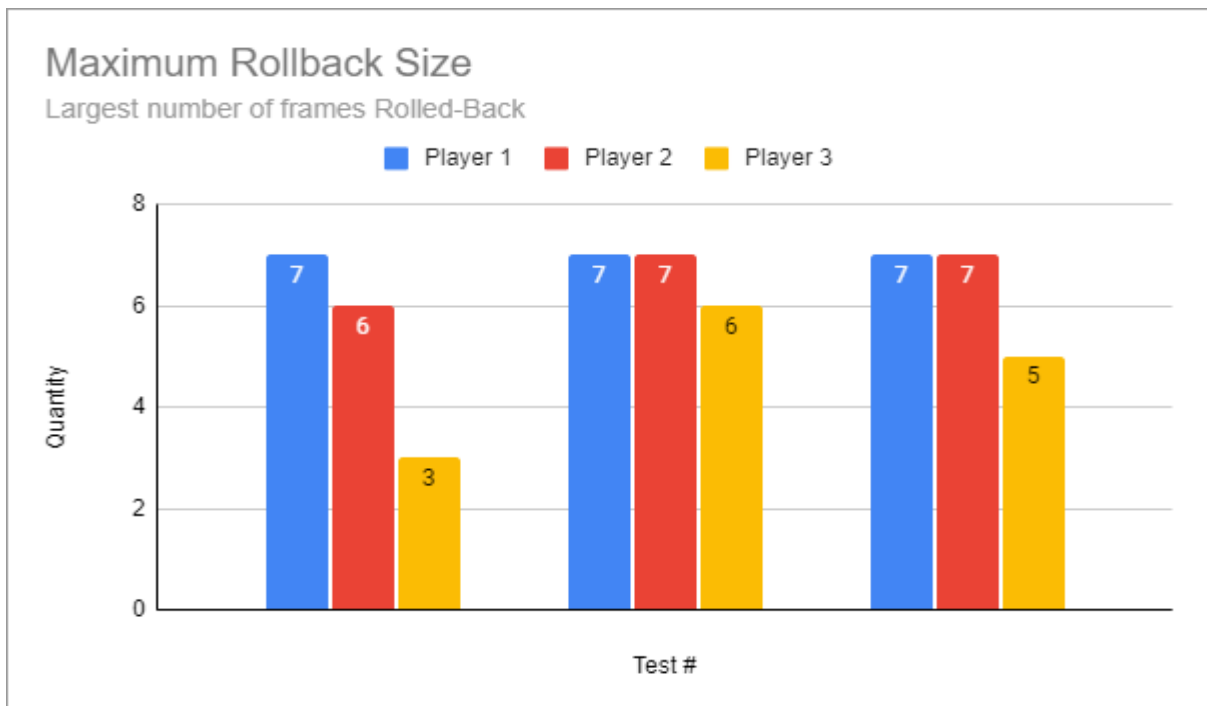


Figure 5 – Largest Single Rollback per Player per Test

Larger latencies necessitate larger Rollbacks to correct past states. Larger Rollbacks then create increased processing overhead during a tick cycle. Note that as the application used a 2-frame input delay, an input on frame 100 would be applied to (and logged for) frame 102, and gives ~32ms of time for receiving packets without needing a Rollback. So under >200ms of lag there should be Rollbacks approaching

the maximum of 10 frames, as $200\text{ms lag} - 32\text{ms} = 168\text{ms}$ late packet, at 16ms per frame it equates to 10.5 frames.

4.3 Unreal Insights Data

The Unreal Insights trace data covers both the UE5 systems and the rendering side of performance. As the trace files contained a lot of irrelevant data, including idle frames of the application's Main Menu before the tests were started, a filter was applied to only look at UE5 render frame timings starting once the internal state saved frame #0, to the last internal state frame before the player quit. As mentioned in the testing methodology, all Unreal Insights data only pertains to Player 3.

Time in MS						
Name	Count	Total Time	Max	Average	Median	Min
Test#1 Frames	12,195	1m 59s	145.69	9.78	9.39	0.6
Test#2 Frames	9,651	1m 54s	44.48	11.85	12.06	0.52
Test#3 Frames	8,416	1m 52s	43.2	13.32	13.04	0.54
Converted to FPS						
Name	Count	Total Time	Max (Lowest)	Average	Median	Min (Highest)
Test#1 Frames	12,195	1m 59s	6.86	102.24	106.49	1666.66
Test#2 Frames	9,651	1m 54s	22.48	84.38	82.91	1923.07
Test#3 Frames	8,416	1m 52s	23.14	75.07	76.68	1851.85

Figure 6 – Unreal Insights Frame Timing Aggregate Data

Given our internal simulation was de-coupled from Unreal's tick rate, it's unsurprising that the rendering is performing at higher speeds than the simulation. Even under perfect conditions, the internal simulation will only tick at 60hz (16.66ms), while the renderer is only restricted by the available processing power.

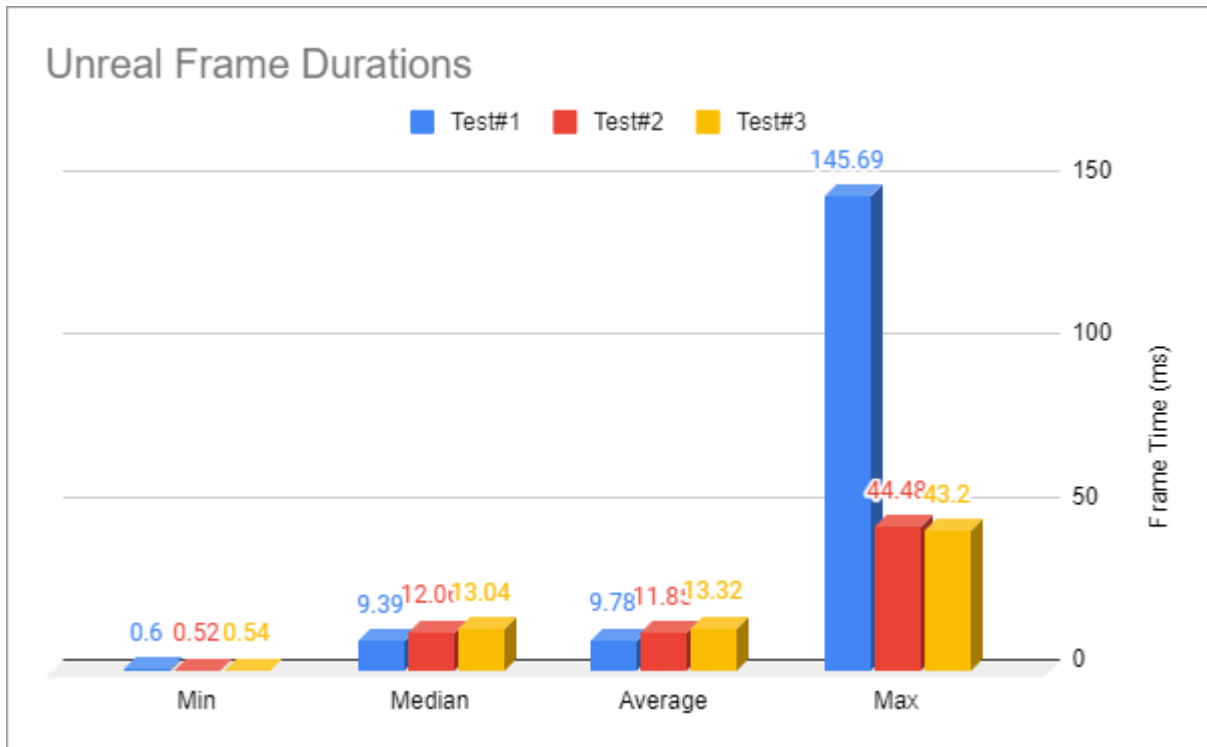


Figure 7 – Unreal Insights Frame Timing Comparison in ms

However the visible spikes in render times, illustrated by the Max times, do demonstrate that the internal simulation can sometimes require increased hardware usage, causing drops in frame rates.

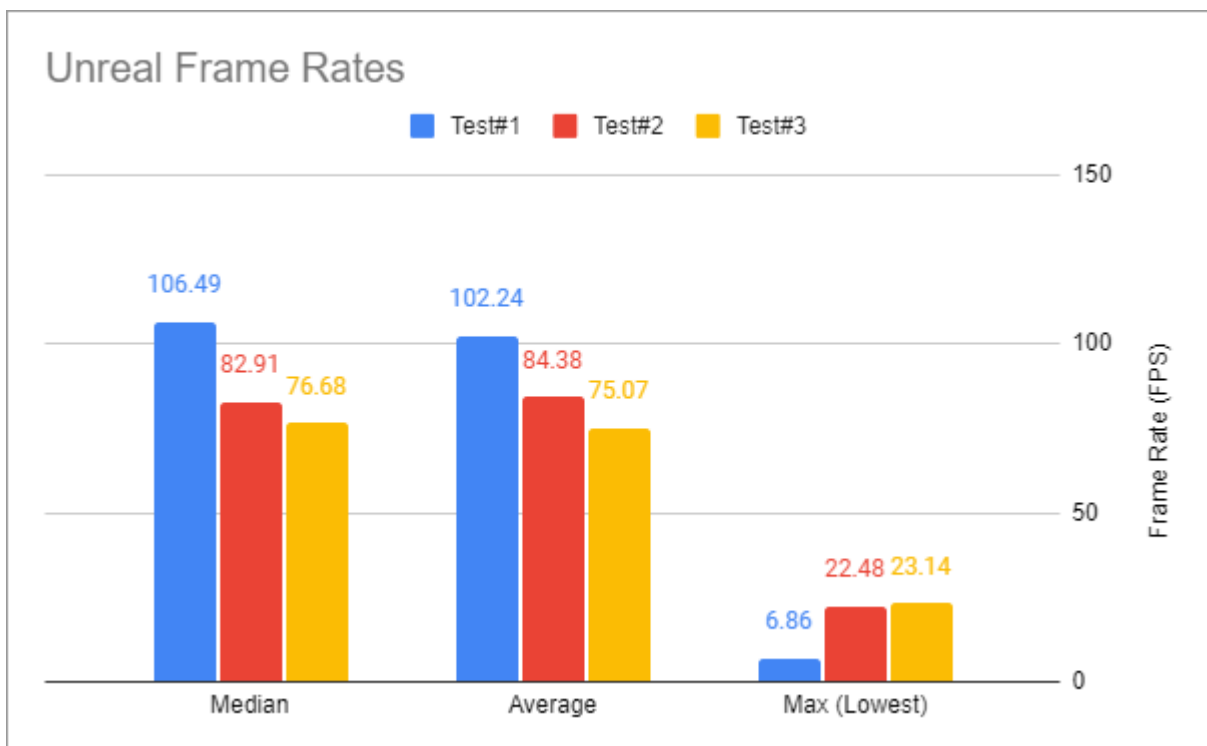


Figure 8 – Frame Timing Comparison upon Conversion to Equivalent FPS

The overall decrease in framerate from test to test also suggest that increased latency (and increased Rollbacks) also affect framerate overall.

5. Discussion

5.1 Overview

To best cover this project's goals, the discussion of the results will be split into two main categories. The first half will focus on interpreting the testing results, with the second half containing a critical retrospective of how implementing Rollback into an FPS affected development.

5.2 Testing Data Analysis

5.2.1 Internal Simulation Performance

Immediately made evident by the Rollback Log data is a noticeable performance problem with the internal simulation. This is indicated by the total frame counts of each test is between 19% and 50% of the target of ~7200 frames for all players. The effects of this were immediately visible during the tests as while the applications were running smoothly according to the visible framerate and Windows Task Manager, the gameplay had obvious stuttering, more so during the later tests.

Performance for Players 1 and 2 was expected to be lower than Player 3, as they had the disadvantage of running simultaneously on a single laptop, but comparing them to Player 3's Rollback data highlights an unexpected consequence. Although Player 3 was by far the least impacted by Rollback's performance costs, by having a fraction of the others' Rollbacks and average rollback size, it had an equivalent total simulation frame count. As this obviously couldn't be a coincidence or related to the processing overhead, this means that GGPO was likely "holding back" the performant player to keep them from outrunning the slowest players. GGPO likely enforces this as it's necessary to avoid a situation where a player could desync by getting 10+ frames ahead (the number of past save states) and suddenly need to Rollback beyond the buffer to re-sync with the others.

Looking at tests with added latency (#2 and #3), we can see that all players have an increase in Rollbacks as expected. However, Player 3, who has the most latency during test #2 and has an equivalently high latency as Player 2 in test #3, is still the least personally impacted by Rollbacks (both in quantity and size). Meanwhile, Player 2 can more closely be compared to Player 1 in impact, which is always the most impacted — despite having no added personal latency in all tests. This suggests that the players with the high latency could be disproportionately affecting other players' game performance more than their own. Hypothetically, there could be a compounding effect created when a problematic player has the highest performance. However, this hypothesis could be a non-issue for applications with better performance that stay on target.

Overall the log data suggests an obvious flaw with this implementation's internal game state performance, but it does point to new findings. All of the relevant literature about performance explains how it must always reach, and then proceed with a variety of suggestions (GDC, 2018; GDC, 2019; GDC, 2021). They are obviously correct in saying this — as the whole point of Rollback was to avoid the stuttering of Lockstep — but this left the question unanswered: what happens when the targets are missed? Concluding this section, the **4.2 GGPO Log data** puts forwards the following answers to that question: having an under-performant player can create a slowdown for other players, players with poor latency can disproportionately increase the other player's performance, and these two forces could be compounding if a player suffering from low-performance is subjected to high-latency opponents.

5.2.2 Unreal Performance

As the **4.3 Unreal Insights Data** could only be collected for Player 3, there is only so much that can be learnt from its analysis. Player 3 had the advantage in performance as it had a laptop to itself and was expected to perform better than the rest. The data suggests that this was indeed the case as Player 3 ran at a higher framerate than the game's internal rate of 60. Unfortunately, it couldn't take advantage of its performance as the gameplay was throttled as previously mentioned.

Given no FPS limits were placed on the renderer, it was allowed to run as fast as possible, unburdened by the gameplay's tick rate. This was fully intended as First-Person-Shooters are not a genre where players want hardcoded FPS limits. It's possible however that this might have had an unknown (but likely minor) impact on the gameplay performance. It's possible that sudden spikes within the simulation, such as a need to do multiple Rollbacks within a small time frame, would run have had more performance headroom if UE5 wasn't simultaneously running at full speed.

Another finding is that despite its independence from gameplay, increasing the latency and the number of Rollbacks does impact UE5 performance. This is the only obvious conclusion from this limited data as Player 3 suffered a 20% to 30% loss in frame rate when testing in the higher latency environments of tests #2/#3. This was somewhat unexpected when considering how Player 3 likely spent more time waiting for others to catch up, which should've increased the average FPS as it had plenty of idle time to render at full speed.

5.3 Reflecting on Rollback Development

While the tests and their data generated many unexpected findings, a large part of this project's aim was to further the understanding of using Rollback outside of its fighting-game niche. This isn't something that can be objectively analysed from any testing data but instead reveals itself when reflecting on the development process. Rollback Networking is a technology that is best described through the metaphor of a "rabbit hole". On the surface, it's simply an evolution of the naive netcode approach of Lockstep, where the massive degradation in online play is solved in an innovative 3-step process. The processes themselves aren't too complex to understand as they're simple enough to make a networking technique a marketable feature (ARCSysWorks 2022). But diving into it reveals the real complexity lies within what is implied in its functionality, namely its requirements.

As evidence of this, almost all of the work involved in the **Literature Review** and **Methodology** sections of this project centres around determinism and internal savable game-states. Because of this, developing within the context of Rollback creates an environment with unique challenges not faced by other types of netcode.

As an example, games using an approach similar to UE5's replication can avoid determinism entirely as the game state information is sent over the network. In that approach, a player's computer is a client that acts as a sort of remote-controlled version of the server's game state, with all actual game code occurring on the server (Unreal Engine, no date). While this could reduce development complexity, it creates new problems that Rollback avoids. The biggest problem with that approach comes from the client only taking action after the server calculates the player input. Because of this, when a player does an input, it can take wildly different times before its effects are displayed, and even once shown the local player is technically viewing the recent past. Meanwhile, Rollback has its static preset input-delay which inherently avoids the random timing issue, and all gameplay on screen is a representation of the true present state. This is likely why the technique was originally adopted in the fighting game genre, as the genre demands extremely fast and precise reaction times from players. Both of those demands just aren't possible with the replication approach once any amount of latency is involved.

This project has accomplished this goal of threading new ground through the other conclusions that come in retrospect. First, its rabbit hole-like qualities create the need for specialisation. As while all aspects of development are impacted by it, there needs to be a specialist who can design the required architecture and development tools to reduce complexity to an individual scale. Second, inherent in this need for specialisation comes a need for increased development resources. A lack of resources means going without the proper specialists, but development without them just creates larger problems down the road. These problems will always demand more resources than what was initially saved, hence why these resources should be considered mandatory. These points are in line with the result of this project, as the scope of the topic is vaster than the limited time and resources available. Evidence of this is the large amount of research required, which then limited the amount of time left for development.

6. Conclusion and Future Work

This project aimed at understanding the principles within Rollback development, exploring what new challenges arise when using it in new genres and evaluating its performance on a game. These elements were successfully addressed throughout the different phases of this project. The principles within Rollback are subjects that require extensive research. Despite it carrying the label of a networking system, it demands a detailed understanding of niche topics that affect every single line of code in a game. New challenges then arose from the conflict between Rollback's requirements and systems used during general game development. Given the all-encompassing nature of requirements like determinism, standard usage popular of game engines and their systems, as you would see in a typical FPS game, cannot be assumed as compatible and must be re-evaluated to fit its mantra. It's vital to have the support of proper Rollback-specific development tools, otherwise, large problems can go unseen and QA testing becomes heavily time-consuming. Finally, testing and data analysis of the performance gave concrete answers to many previously unexplored questions throughout the discussion of the results.

The implications of the results are the following. Rollback is best to consider in early development or pre-production, as it will dictate future decisions and is a lot harder to retroactively implement. Next, Rollback brings with it strengths and weaknesses, its strengths complement games with fast-paced competitive natures, while its weaknesses are the amount of work needed to support it. This means that gameplay design philosophy should decide if Rollback is a suitable fit; and if a game doesn't play to its strengths, efforts might best be put elsewhere.

6.1 Improvements to be Made and Future Work

Concluding this project accentuates the improvements to be made in any future development of this project. First, additional tools for desync detection and automatic performance analysis are required. An occurrence of either of these problems immediately destroys gameplay; so to even approach what seems like mandated perfection, implementation tools that save time by quickly identifying the source of a problem will improve both development and the end product. Second, this implementation has unidentified problems with determinism likely due to the attempt

at serialisation of Bullet Physics. Any new attempt should immediately address this issue, either with an improved methodology or a different approach to using physics entirely. Finally, it's best to approach the topic with more resources than this attempt, as seniors with specialisation in the field, alongside a larger team, would make the process infinitely easier and more thorough.

For future work or research, there are two key areas to address. This project wouldn't have been anywhere near possible if it wasn't for the existence of open-source or free-to-use tools such as the GGPO Rollback Networking SDK, Bullet3's Real-Time Physics Simulation, Unreal Engine 5, and prior work in the field of integrating Rollback into Unreal by BwdYeti's port of GGPO and Vector War into UE4. This is the first direction this project recommended: creating new and improved tools that allow new developers, who might lack the resources to create proprietary tools, the ability to take advantage of Rollback. Other than that, an angle unexplored by the current implementation of Rollback would be using it at a greater scale through high-performance servers or cloud computing. Current implementations of Rollback have mostly restricted themselves to Peer-to-Peer 2 to 4-player games due to the increased processing power demanded by each additional player (as there are more instances requiring rollbacks). It would be interesting to see if it would be possible to achieve high player counts, such as in Battlefield series or MMOs by taking advantage of new types of computing power.

List of References

ARCSysSystemWorks (2020) [PRESS RELEASE] *Guilty Gear -Strive- Closed Beta Test Sign-ups Start Today*. Available at: <https://www.arcsystemworks.com/press-release-guilty-gear-strive-closed-beta-test-sign-ups-start-today/> (Accessed 18 October 2022).

ARCSysSystemWorks (2022) *ROLLBACK is coming to #GUILTYGEAR Xrd -Revelator- and GUILTY GEAR Xrd REV 2!* Available at: <https://twitter.com/ArcSystemWorksU/status/1574195918294892545?s=20&t=qE72Ardn8Y8Pw1owEaAmEA> (Accessed 18 October 2022).

BwdYeti (2020) *GGPOUE4*. Available at: <https://github.com/BwdYeti/GGPOUE4> (Accessed 22 November 2022).

BwdYeti (2020) *VectorWarUE4*. Available at: <https://github.com/BwdYeti/VectorWarUE4#readme> (Accessed 22 November 2022).

Cannon, T. (2019) *GGPO GitHub Documentation*. Available at: <https://github.com/pond3r/ggpo/tree/master/doc> (Accessed 18 October 2022).

Clumsy (no date) *clumsy 0.2*. Available at: <https://jagt.github.io/clumsy/index.html> (Accessed 21 May 2023).

D'Anastasio, C. (2018) 'Super Smash Bros. Still Can't Get Online Right [UPDATE]', *Kotaku*, 10 December. Available at: <https://kotaku.com/super-smash-bros-still-cant-get-online-right-1830993555> (Accessed 18 October 2022).

Dawson, B. (2013) 'Floating-Point Determinism', *Random ASCII – tech blog of Bruce Dawson*, 16 July. Available at: <https://randomascii.wordpress.com/2013/07/16/floating-point-determinism/> (Accessed: 23 November 2022).

De Ioris, R. (2021) 'Game Engines and Determinism', *Duality Robotics Blog*, 18 August. Available at: <https://www.duality.ai/blog/game-engines-determinism> (Accessed: 22 November 2022).

Diler, L., Dalil, S., and Mota, M. (2022) 'Hybrid Prediction for Games' Rollback Netcode', *Special Interest Group on Computer Graphics and Interactive Techniques Conference Posters*, ACM SIGGRAPH (2022 Posters), Article No. 3, p. 1-2, doi: 10.1145/3532719.3543199

Fiedler G. (2004) *Fix Your Timestep!* Available at: https://gafferongames.com/post/fix_your_timestep/ (Accessed 30 May 2023).

The Game Awards (2014 to 2022). Accessible at the same URL with different years in the end: https://en.wikipedia.org/wiki/The_Game_Awards_2014 (Accessed May 29, 2023).

GDC (2018) *8 Frames in 16ms: Rollback Networking in 'Mortal Kombat' and 'Injustice 2'*. Available at: <https://www.gdcvault.com/play/1025021/8-Frames-in-16ms-Rollback> (Accessed 18 October 2022).

GDC (2019) *Back to the Future! Working with Deterministic Simulation in 'For Honor'*. Available at: <https://www.gdcvault.com/play/1026322/Back-to-the-Future-Working> (Accessed 18 October 2022).

GDC (2021) *Breaking the Ankh: Deterministic Propagation Netcode in 'Spelunky 2'*. Available at: <https://www.gdcvault.com/play/1027358/Breaking-the-Ankh-Deterministic-Propagation> (Accessed 18 October 2022).

GGPO (no date) GGPO Rollback Networking SDK. Available at: <https://www.ggpo.net> (Accessed 18 October 2022).

GGPO (2020) *Is there a maximum size in bytes for an input when creating a GGPOSession? #54*. Available at: <https://github.com/pond3r/ggpo/issues/54> (Accessed 16 May 2023).

Wikipedia (no date) *GGPO*. Available at: https://en.wikipedia.org/wiki/GGPO#Games_using_rollback_networking (Accessed Feb 10 2023)

Portelli G. (2016) *Everything you always wanted to know about Unreal Engine physics (but were afraid to ask)*. Available at: http://www.aclockworkberry.com/unreal-engine-substepping/#footnote_6_530 (Accessed 20 May 2023)

Hold Back to Block (2019) *Talking Netcode With Adam "Keits" Heart*. Available at: <https://youtu.be/1RI5scXYhK0> (Accessed 18 October 2022).

Infil (2019) *Fighting Words: Netcode, Explaining how fighting games use delay-based and rollback netcode*. Available at: <https://words.infil.net/w02-netcode.html> (Accessed 18 October 2022).

Lee W. K. S. and Chang K. C. R., (2017), "On "shot around a corner" in first-person shooter games", *2017 15th Annual Workshop on Network and Systems Support for Games (NetGames)*, pp. 1-6, doi: 10.1109/NetGames.2017.7991545.

NVIDIA (no date) *PxSceneFlag Struct Reference*. Available at: <https://docs.nvidia.com/gameworks/content/gameworkslibrary/physx/apireference/files/structPxSceneFlag.html> (Accessed 20 May 2023).

Orland, K. (2011) "Interview: How A Fighting Game Fan Solved Internet Latency Issues", *Game Developer*, 22 April. Available at: <https://www.gamedeveloper.com/business/interview-how-a-fighting-game-fan-solved-internet-latency-issues> (Accessed 18 October 2022).

PyBullet (2020) *bullet3/examples/pybullet/examples/saveRestoreState.py*. Available at: <https://github.com/bulletphysics/bullet3/blob/master/examples/pybullet/examples/saveRestoreState.py> (Accessed 25 April 2023).

Streeting S. (2020) *Using Bullet for Physics in UE4*. Available at: <https://www.stevestreeting.com/2020/07/26/using-bullet-for-physics-in-ue4/> (Accessed 28 Feb 2023).

Unreal Engine (no date) *bEnableEnhancedDeterminism*. Available at: <https://docs.unrealengine.com/5.1/en-US/API/Runtime/Engine/PhysicsEngine/UPhysicsSettings/bEnableEnhancedDeterminism/> (Accessed 28 May 2023).

Unreal Engine (no date) *Networking Overview*. Available at: <https://docs.unrealengine.com/5.1/en-US/networking-overview-for-unreal-engine/> (Accessed 28 May 2023).

Unreal Forums (2016) *Using a fixed physics timestep in Unreal Engine, free the physics approach*. Available at: <https://forums.unrealengine.com/t/using-a-fixed-physics-timestep-in-unreal-engine-free-the-physics-approach/67537> (Accessed 30 May 2023)

Yu D. (2019) *Tweet About Spelunky 2's Game Engine*. Available at: <https://twitter.com/mossmouth/status/1175118914407108608?s=20> (Accessed 30 May 2023)

Bibliography

- https://www.reddit.com/r/RocketLeague/comments/8m1aed/netcode_lag_explained_with_lots_of_examples/
- https://www.reddit.com/r/unrealengine/comments/j4ju8k/finally_got_custom_multiplayer_physics_working/
- <https://forums.unrealengine.com/t/physics-simulation-fixed-time-step-and-enhanced-determinism-settings-pull-request/93309>